

Optimus-Cost-Agent

Test Strategy

Validation Plan - Phase 1, Sprint 1 - Gateway-Centric Local Runtime

Version 1.6

Architected by: Vibhanshu Agarwal

1. Test objectives

The strategy proves that full Plan and Agent runs resolve only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY` in the agent process. The agent resolves zero upstream credentials. The loopback Gateway may hold its approved model-aggregator credential plus multiple operator-provisioned, profile-scoped MCP credentials; no Gateway-held secret crosses to the agent, ACP child, logs, telemetry, state, responses, or error text.

Gateway-brokered MCP is tested as a tools-only, static-profile capability with remote Streamable HTTP and Docker-contained stdio transports. The test objective is not merely agent-cage behavior: every approved MCP invocation must prove both the Plan 6.5 agent gate and the independent Gateway profile/binding/allowlist/freshness/resource/budget gate.

2. Scope and non-scope

In scope are typed `discover` and `call` routes, registration/refresh, restart activation, namespace collisions, descriptor filtering and context admission, complete pagination, both transport profiles, result validation, attribution-aware MCP accounting, indeterminate holds, and real cross-platform containment evidence.

Out of scope are dynamic OAuth/device flow (P11-FU-12), non-tool protocol capabilities and multi-round interaction (P11-FU-13), semantic tool search/code mode (P11-FU-15), ACP client-supplied `mcpServers` (P11-FU-9), and ACP session-resume/session-load custody (P11-FEAT-ZED-RESUME). These are distinct capabilities with separate reasons and must not be silently represented by Gateway-brokered MCP evidence.

3. Test pyramid and evidence tiers

Tier	Dependency	Fakes permitted	Claim supported
Unit	In-process functions	Yes	Local logic and contracts only
Contract	Recorded request/response shapes	Yes	Parser and schema behavior only
requires_redis	Real TimeSeries-capable Redis	No	Persistence and retention
requires_gateway	Real loopback Gateway + approved credentials	No	Gateway behavior
requires_mcp_stdio	Real Gateway + independently authored Docker-contained stdio MCP server	No	Stdio interoperability and containment
requires_mcp_http	Real Gateway + independently authored Streamable HTTP MCP server	No	Remote HTTP protocol/ freshness behavior
ACP protocol	Independent acpx client	No	Real ACP client compatibility
E2E / Release	Spawned ACP process + named real dependencies	No	Golden workflow / Phase 1 sign-off

Fakes are unit-tier only. A fake, a project-authored MCP server, or a different HTTP server cannot justify a live MCP claim. Platform confinement additionally requires real Windows and Linux/WSL2 evidence; a green remote CI result alone does not establish the platform-gated control.

Context7 is a named remote-compatibility dependency. Before any Context7 reachability claim, the configured endpoint receives a live authenticated Gateway-originated server/discover probe proving 2026-07-28 and tools support. Unsupported or indeterminate evidence is fail-closed with `mcp.protocol_version_unsupported`; a fake or another HTTP server cannot discharge this dependency.

LLD Section	Design Claim	Test Category (\$)	Tier
\$4A — State Machine	AwaitingApproval gate cannot be bypassed	\$6 — Mode/State	Unit + Integration
\$6 — Gateway Layer	POST /v1/responses uses input field, not messages	\$8 — Gateway	Unit + Integration
\$6 — Gateway Layer	No provider key present in local environment during run	\$8 — Gateway	E2E (Release Gate)
\$9 — Tool Policy	EvidenceRequest.query sent verbatim (not substituted)	\$7 — Tools	Unit
\$9 — Tool Policy	EvidenceExtractRequest rejects out-of-set URLs	\$7 — Tools	Unit
\$9 — Evidence Ledger	total_cost_usd() reconciles against gateway response	\$9 — Cost	Unit + Integration
\$9D — Gateway Policy	Gateway revalidates domain allowlist independently	\$8 — Gateway	Integration
\$10 — Telemetry	RedisTimeSeries TS.CREATE idempotent with TS.ALTER fallback	\$10 — Telemetry	Integration
\$11 — Sprint 1 Gate	One-key setup: full run with only OPTIMUS_* env vars	\$12 — Release Gate	E2E

5. Mode and State Transition Tests

LLD reference: §4 Behavioral Governance, §4A Agent State Model. These tests prove the state machine is deterministic and that no path bypasses the mode enforcement primitives.

Unit Tests — `assert_mutation_allowed()`

- ☐ In Plan/Chat mode: calling `assert_mutation_allowed()` raises Code -32002 with message "mutation forbidden in Plan/Chat mode".
- ☐ In Agent mode before AwaitingApproval: `assert_mutation_allowed()` raises Code -32002.
- ☐ In Agent mode after approval granted: `assert_mutation_allowed()` returns None (no exception).
- ☐ Calling any mutation tool (`write_file`, `shell_exec`, `shadow_apply`) in Plan/Chat mode raises -32002 before any I/O occurs.

Unit Tests — State Transitions

- ☐ Idle → Planning: valid on any user request.
- ☐ PlanReady → Executing (direct): rejected with -32002; must pass through AwaitingApproval.
- ☐ AwaitingApproval → ChatOnly on timeout: confirmed after configurable `timeout_ms` expires.
- ☐ Failed → Planning: retry counter increments; failure context injected into next prompt payload.
- ☐ Failed → Terminated: after `max_retries=3` consecutive gate failures, state transitions to Terminated and `user_escalation` signal emitted.

Integration Tests — Mode Boundary

- ☐ Full Plan/Chat run: agent completes discussion and returns plan text; no file in working tree is modified; telemetry records 0 mutation calls.

6. Tool Invocation and MCP Integration Tests

Existing search, extract, package, and advisory tests remain unchanged. MCP adds the following real-Gateway tests for both transport profiles:

- registration bootstrap accepts only a pending/stale `profile_id` and revision, performs discovery, obtains operator approval, then proves restart-to-activate before refresh or call;
- a direct shared-bearer call cannot exceed the Gateway allowlist even when agent-only scope/effect checks are bypassed; detected binding drift denies while recoverable refresh returns only the prior approved manifest with Gateway-side freshness `stale_marked`;
- unmatched upstream allowlist entries are reported, two profiles with the same upstream name remain distinct as `profile_id`, `tool_name`, and canonical descriptor hashes match across agent/Gateway;
- descriptor filtering excludes off-allowlist and invalid `x-mcp-header` definitions, never emits `Mcp-Param-*`, preserves exact descriptor text for scanning, and records descriptor-context selected identities plus count/byte ceilings;
- tool arguments are the only agent-originated payload; system prompt, conversation history, policy, approval data, and credentials never reach the MCP server.

Remote HTTP tests assert per-request `_meta`, `server/discover`, `tools/list`, and `tools/call` at 2026-07-28, absence of HTTP `initialize/session/ping/standalone` GET behavior, redirect denial, and POST-SSE duration/byte bounds. Missing remote discovery/version/tools support yields only `mcp.protocol_version_unsupported`, never HTTP fallback.

7. Gateway Settings, Transport, and Egress Gates

The agent and ACP child may egress only to the loopback Gateway. The Gateway may egress only to its approved model/tool hosts and the provisioned MCP endpoint. Agent and Gateway environments, child environment, logs, traces, state, response, and error text are scanned separately for all upstream credentials.

Containerized stdio proves discovery-first negotiated/legacy tools-only behavior. The enforced tier requires a digest-pinned Docker image, no host mounts/devices/Docker socket, safe docker run --env NAME projection, timeout/output limits, bounded reads, and deterministic termination. The platform-gated process-count control requires real Windows Job Object and Linux/WSL2 evidence. Docker daemon/image trust and allowed container network egress remain residuals; neither MCP roots nor the deferred model-generated code-execution pattern is a sandbox boundary.

7.1 Discovery, cache, and lifecycle tests

tools/list pagination is complete-or-absent: fixtures prove ordered accumulation until no nextCursor, while repeated/malformed cursors or malformed pages deny without an approvable prefix. Only transient server/discover/tools/list failures retry; capacity exhaustion yields the narrow no-manifest disposition. Tests prove ttLMs caps freshness, cache entries are partitioned by profile revision and credential binding, and transport open/close cannot activate, re-enable, or mutate a profile.

8. Cost Accounting Tests

LLD reference: §10 Usage Accounting Service, §9 Evidence Ledger. These tests prove that cost accounting is accurate, complete, and reconciles against the gateway response.

Unit Tests — EvidenceLedger

- ☐ `total_cost_usd()` sums `cost_usd` across all `EvidenceLedgerEntry` objects; returns 0.0 on empty ledger.
- ☐ `total_billing_units()` sums `billing_units`; matches provider-native unit count.
- ☐ `total_credits()` (backward compat) sums `credits_used`; does not conflict with `cost_usd` total.
- ☐ GatewayUsage fields (`gateway_request_id`, `provider`, `provider_request_id`, `cache_hit`, `billing_units`, `cost_usd`) all propagate correctly from gateway response envelope to `EvidenceLedgerEntry`.
- ☐ Ledger entries are append-only; no existing entry can be modified after `record()` is called.

Unit Tests — Pricing Fallback

- ☐ Missing `pricing_snapshot.json`: usage accounting engine falls back to default pricing matrix and raises telemetry audit signal `PRICING_FALLBACK_ACTIVATED`.
- ☐ `pricing_snapshot.json` present but stale (> 24h): `PRICING_SNAPSHOT_STALE` audit signal raised; calculation proceeds with stale data.

Integration Tests — RedisTimeSeries

- ☐ `TS.CREATE` with retention 30 days: key created; `RETENTION` verified via `TS.INFO`.
- ☐ Duplicate `TS.CREATE` on existing key: `TS.ALTER` applied idempotently; `RETENTION` updated; no error thrown.
- ☐ `TS.ADD` with `run_id` tag: confirmed present in `TS.RANGE` query with tag filter.
- ☐ Run metadata hash (HSET): `execution_mode`, `generation_scope`, `rigor_level`, assumption ledger count all written at workflow completion; TTL set to 30 days.

8A. Test Coverage Target, Measurement & Trace Observability

This document is the authoritative source of truth for the Phase 1 coverage target, the measurement-tool taxonomy, and trace observability. The HLD (§12, §11A) and LLD (§11A) may **summarize** the target for context, but Test Strategy §8A remains authoritative; where any summary diverges, this section governs.

Test Coverage Target and Measurement

Phase 1 targets a minimum **80% aggregate Python test coverage** threshold for production code, measured in CI with `coverage.py` and surfaced through `pytest-cov` during normal `pytest` execution. The 80% threshold is a release gate, not a quality substitute: deterministic safety-critical modules such as `ToolRegistry` authorization, `MutationGuard`, `GatewayUsage` accounting, `EvidenceLedger` reconciliation, JSON-RPC framing, retry/fail-closed logic, and policy enforcement should trend materially higher and must not regress without explicit review.

`Coverage.py` is the canonical coverage engine for line and branch coverage reporting; `pytest-cov` is the preferred `pytest` integration for local and CI workflows. Agent-quality and AI-safety evaluations

8. Cost Accounting Tests and §8A Consumer Sweep

Legacy settled GatewayUsage and ProviderUsage rows remain byte-compatible. A separate MCPUsageRecord is tested across persistence, current-run budget enforcement, display, reconciliation, telemetry, Redis schemas, evidence ledger, and golden-task consumers.

Attribution state	Required evidence
settled	authoritative billing units and cost_usd survive round trip and reconciliation
explicit_zero	exactly zero only with a revision-bound operator declaration of free external charge
unavailable	absent monetary fields, never zero display/reconciliation, strict-budget pre-dispatch denial unless policy permits

The consumer sweep proves no legacy settled row changes shape or semantics. It proves `mcp.budget.unattributed_spend_denied`, unattributed display/reconciliation behavior, and that a result is withheld and the run held until the same `gateway_request_id` persists. Recovery retries persistence only; it never redispaches tools/call.

8A. Trace and evidence assertions

Every MCP record carries run/session/request correlation, profile/revision, namespaced/upstream tool, transport, disposition, request/response bytes, duration, attribution state, and redacted diagnostics. Trace assertions include pre-tool/Gateway decision, freshness, result disposition, retry outcome, accounting persistence, and final release/hold. External MCP logging is closed: no logging request or channel may feed, change, or suppress Optimus append-only audit logging or telemetry.

9. Error, Retry, and Failure Injection Tests

MCP errors extend the existing `RetryPolicy`; they do not create a parallel retry engine. `tools/call` never retries automatically. Only transient `server/discover` and `tools/list` failures use existing exponential backoff with jitter, at most three attempts, and a restarted complete scan. Authorization drift, pagination integrity, policy, schema, accounting, and result-validation failures are permanent.

Timeout or loss after dispatch is indeterminate. A read-only tool may be invoked again; a side-effecting (`profile_id`, `tool`) holds for explicit operator acknowledgment before and after agent restart. The durable hold does not assert that the prior attempt failed.

10. Schema and result validation tests

Fixtures prove `resultType`: `complete` is the only releaseable result, `structuredContent` is checked against `outputSchema`, and invalid schema, oversize output, timeout, stale binding, or invalid profile state fails closed. `resource_link` and embedded resources remain inert; automatic dereference, image/audio promotion, decoding, persistence, execution, or policy mutation is denied with a typed disposition.

`input_required`, `roots`, `sampling`, `elicitation`, `subscriptions`, `unsolicited server requests`, `external logging`, and `unsupported methods` are denied at `method/result/content` boundaries. No denial turns into a planner question, profile transition, or implicit retry.

11. Security and Trust Boundary Tests

Security fixtures prove descriptor/result distrust, no auto-load from cloned repositories, manifest reapproval after profile or descriptor change, and argument-only payload isolation. Catalog metadata may prefill a pending operator proposal only; it never causes registry lookup, install, connect, activation, trust inheritance, agent/model lookup, or Gateway data-plane query.

The split-agency matrix proves the Plan 6.5 human approval, descriptor trust, permission scope, and effect-class checks remain distinct from the Gateway profile/binding/allowlist/resource/budget checks. Direct bearer misuse is bounded by the Gateway allowlist; no test claims that this protects against a compromised Gateway.

Elicitation fixtures expose no form schema, URL, or opaque request state to the planner. Sampling fixtures prove no model call, budget reservation, provider usage, MCPUsageRecord, server response, or inherited context in v1. If sampling is later opened, tests require a linked record and two human decisions: one before budget reservation and one before returning model output to the server.

Generalized OWASP reference-only rows in HLD/LLD create no acceptance criteria. Only separately labelled NORMATIVE – P11-FEAT-GATEWAY-MCP controls map to these test obligations.

12. Golden Task Regression Suite

MCP golden tasks run against the real loopback Gateway and independently authored stdio and Streamable HTTP servers. Each records expected mode, tool/authorization outcome, profile/revision/binding, namespaced identity, transport, cost band, provider/MCP usage, mutation behavior, request identities, trace identity, and final disposition.

The suite covers pending registration, restart activation, direct-bearer allowlist denial, descriptor drift/reapproval, stale-marked refresh, namespace collision, invalid header filtering, complete pagination/cursor denial, remote unsupported version, stdio containment, result/resource rejection, unknown-cost denial, persistence-only recovery, indeterminate acknowledgment, and closed capability fixtures. A fake cannot stand in for the named Gateway or MCP server dependency.

Process-scope assertion

The agent has zero upstream credentials. Gateway profile credentials remain process-scoped and cannot appear in agent/child environment, logs, telemetry, state, responses, or error text. This assertion is separate from the model-aggregator credential and from MCP attribution/accounting evidence.

13. Phase 1 Release Gates

Transport and protocol

- Real Gateway plus independently authored stdio and Streamable HTTP MCP server evidence passes.
- Remote HTTP proves 2026-07-28 per-request metadata, server/discover, tools support, and no HTTP fallback; stdio proves discovery-first modern/legacy tools-only negotiation.
- Context7 reachability is not claimed until the configured endpoint passes the authenticated Gateway-originated exact-version/tools probe; unsupported/indeterminate HTTP evidence fails closed.

Security, containment, and cost

- Agent zero-upstream-credential scans, Gateway allowlist/binding/freshness checks, and argument-only payload tests pass.
- Enforced Docker containment and platform-gated Windows/Linux process evidence pass; daemon/image/ network residuals remain recorded.
- MCP accounting consumer sweep, strict unknown-cost denial, durable holds, persistence-only recovery, and no automatic tools/call retry pass.

Closed capability and evidence boundary

- input_required, roots, sampling, elicitation, logging, subscriptions, unsolicited server requests, automatic resource dereference, and image/audio promotion are fail-closed without changing Optimus audit logging, profile state, or budget.
- Release sign-off remains governed by the authoritative Plan 9.6 live-verification gate.

14. Guardrail & Workflow Test Cases

LLD reference: §12. Companion reference: Guardrails v1.2 §11. Each control traces to an executable test category; a control without a category is not validated for Phase 1.

14.1-14.3 Existing permission and shell controls

Permission deny precedence, mode short-circuit, impact holds, shell validator, and Unicode/homoglyph tests remain unchanged.

14.4 Prompt-injection fixture tests — extended

Poisoned agent config and poisoned MCP tool metadata are caught by ConfigTrustScanner / MCPTrustRegistry on ingest. Extension fixtures use exact upstream descriptor text, versioned canonicalization, filtered allowlist output, and namespaced identity. They prove tool descriptions, schemas, result content, and annotations cannot modify policy, widen scope, trigger dereference, or become trusted manifest content.

14.5 MCP autoload denial tests — extended

A server bundled in a cloned repository does not auto-load; manifest-hash or profile-revision change forces reapproval; `allowed_tools` are enforced. Catalog metadata may only prefill a pending operator proposal. It cannot cause lookup, installation, connection, activation, or runtime trust. Tests keep Gateway-brokered MCP distinct from ACP `mcpServers` (P11-FU-9) and ACP session resume (P11-FEAT-ZED-RESUME).

14.6-14.9 Existing parity, bypass, loop, and skills controls

Pre-commit/CI parity, bypass, bounded-loop, and skill loading/trust tests remain extended only by the MCP rows below; no generic reference-only security statement becomes a new acceptance criterion.

14.10 MCP additions to the Requirements-to-Test Traceability Matrix

Design evidence claim	Required executable evidence	Tier
Agent has no upstream MCP credential	process-specific environment, keyring/config, log, and egress scans	E2E + Release
Direct bearer cannot widen tools	real Gateway direct-route allowlist denial	requires_gateway
Profile changes / rotation force reapproval	profile-surface matrix and successor to test_launch_env_change_forces_reapproval_without_logging_secret_values proving no secret or secret-derived digest is logged/returned	Integration + E2E
Bootstrap and namespace are correct	pending discovery, approval, restart activation; collision-safe profile_id.tool_name	requires_gateway
Discovery is bounded and fresh	complete nextCursor, cursor-integrity denial, transient/capacity distinction, ttlMs, partitioned cache, stale marker versus drift denial	requires_mcp_http + requires_mcp_stdio
Remote/stdio protocol behavior is correct	remote 2026-07-28 tools-only request metadata/no fallback; stdio discovery-first negotiated/legacy behavior	both live MCP tiers
Descriptor and payload boundary holds	allowlist/filtering, x-mcp-header, no Mcp-Param-*, exact scan, context count/bytes, arguments-only	Unit + Integration
Result and deferred features fail closed	complete-only/schema, inert resources, image/audio, input_required, roots, sampling, elicitation, logging/subscriptions	Unit + Integration
Stdio isolation is real	Docker enforced tier and Windows/Linux platform-gated evidence; residuals remain explicit	requires_mcp_stdio + Release
Unknown spend is never zero	MCPUsageRecord states, strict denial, display/reconciliation, consumer sweep	Unit + requires_redis
Indeterminate mutation is not repeated	durable side-effect hold/acknowledgment across agent restart; read-only re-invocation	Integration + E2E
Retry and accounting recovery are bounded	no automatic tools/call retry; discovery retry cap; result withholding and same-ID persistence-only recovery	Integration
Catalog and connection axes stay separate	no registry action from metadata; open/close cannot mutate profile	Unit + Integration
Context7 claim is honest	configured real endpoint receives authenticated Gateway probe for exact version plus tools	requires_mcp_http + Release

The matrix maps only NORMATIVE – P11-FEAT-GATEWAY-MCP controls. It does not create acceptance criteria from HLD/LLD REFERENCE – Cross-cutting material.